Name: Arya Rane
Roll no.: 2401090
Batch: S21

Experiment 3

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Aim: To deploy multi-service applications using Docker Compose.

Theory:

Docker Compose is a powerful tool used to define, configure, and manage multi-container Docker applications in a simple and efficient way. In a traditional approach, each container must be created and managed separately using multiple docker run commands, which can become difficult when an application requires several interconnected services. Docker Compose solves this problem by allowing developers to describe the complete application architecture in a single configuration file called docker-compose.yml. This file contains all the necessary information about application services, Docker images, container ports, shared networks, persistent storage volumes, and environment variables required for running the application.

A multi-service application is an application that consists of two or more containers that work together to provide complete functionality. For example, a modern web application may include a frontend container responsible for the user interface, a backend container that handles business logic and API requests, and additional services such as databases or caching systems. Docker Compose automatically creates a dedicated network for these services, allowing containers to communicate securely with each other using their service names instead of manually configured IP addresses.

During deployment, Docker Compose reads the instructions provided in the docker-compose.yml file, builds the required Docker images, creates and configures containers, establishes communication networks, attaches storage volumes, and starts all application services with a single command:

Advantages of Docker Compose

- Simplifies deployment of multi-container applications.
- Uses a single configuration file for all services.
- Automatically creates a network for container communication.
- Supports persistent storage using volumes.

Name: Arya Rane
Roll no.: 2401090
Batch: S21

- Makes development, testing, and deployment consistent across different environments.
- Reduces manual configuration and improves maintainability.

Applications

- Web applications with backend and database services.
- Microservices-based applications.
- Development and testing environments.
- Continuous Integration and Continuous Deployment (CI/CD) workflows.

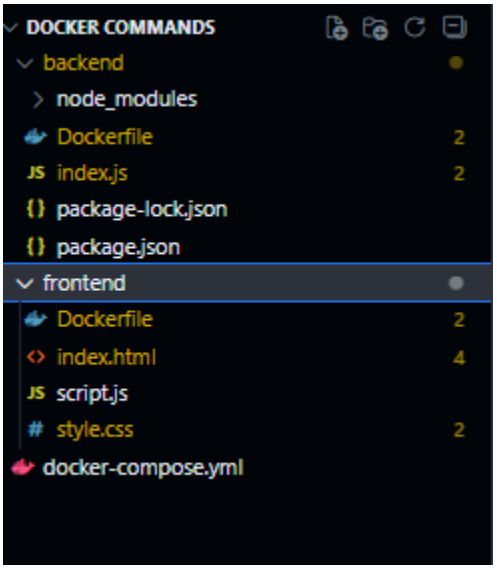
Implementation:

```
C:\Users\15L>docker compose version
Docker Compose version v2.24.6-desktop.1

C:\Users\15L>docker --version
Docker version 25.0.3, build 4debf41
```

Step 1: Create the Project Directory

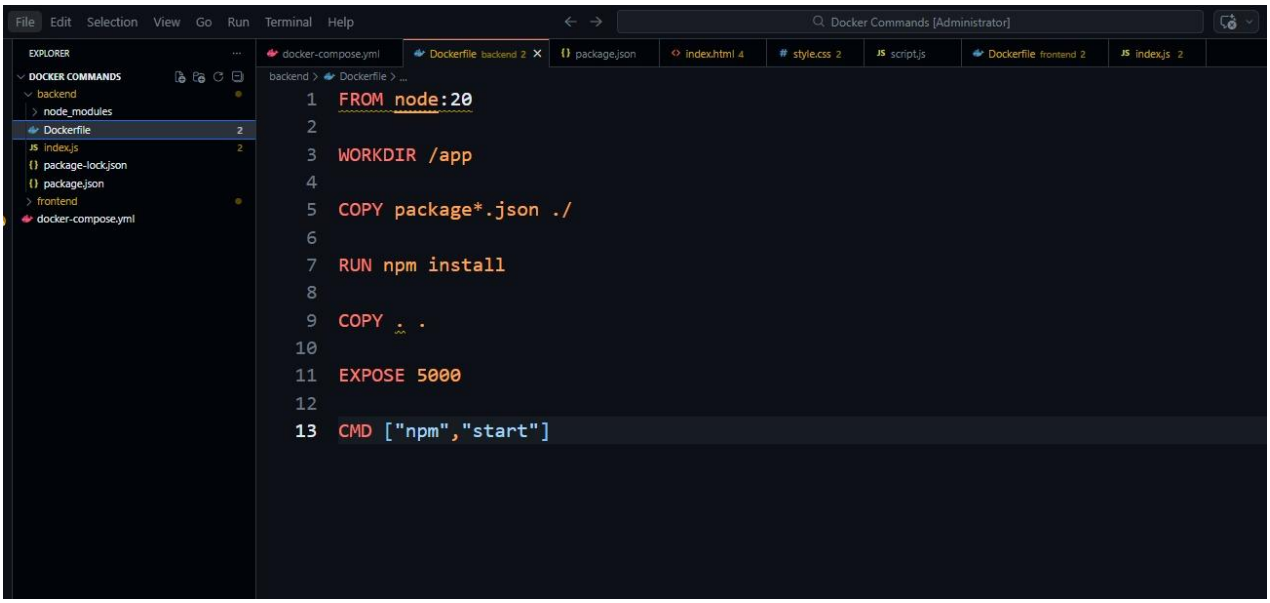
File structure:



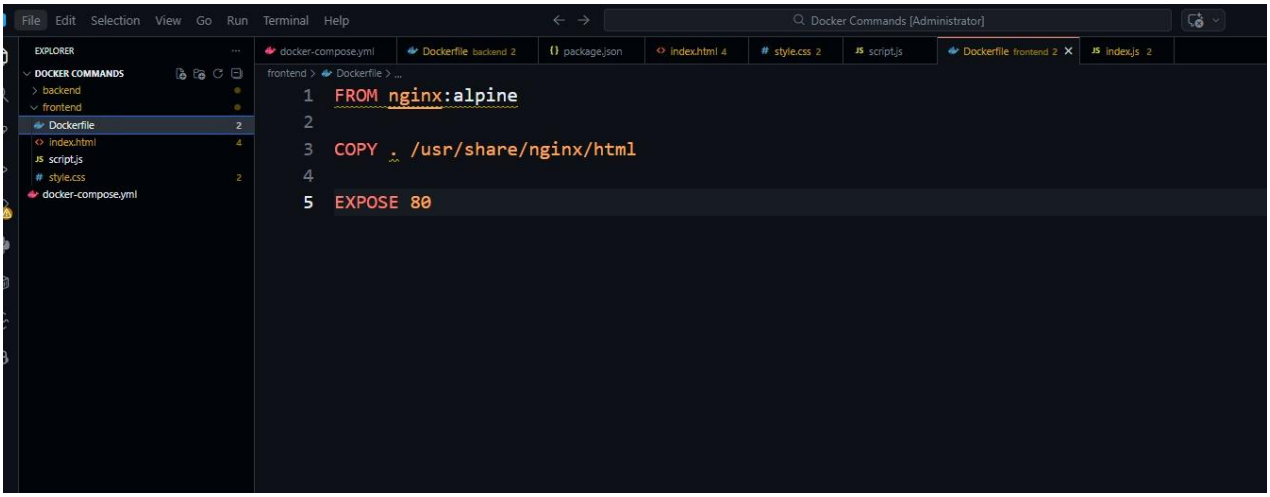
Name: Arya Rane
Roll no.: 2401090
Batch: S21

Step 2: create docker file

Backend/Dockerfile:

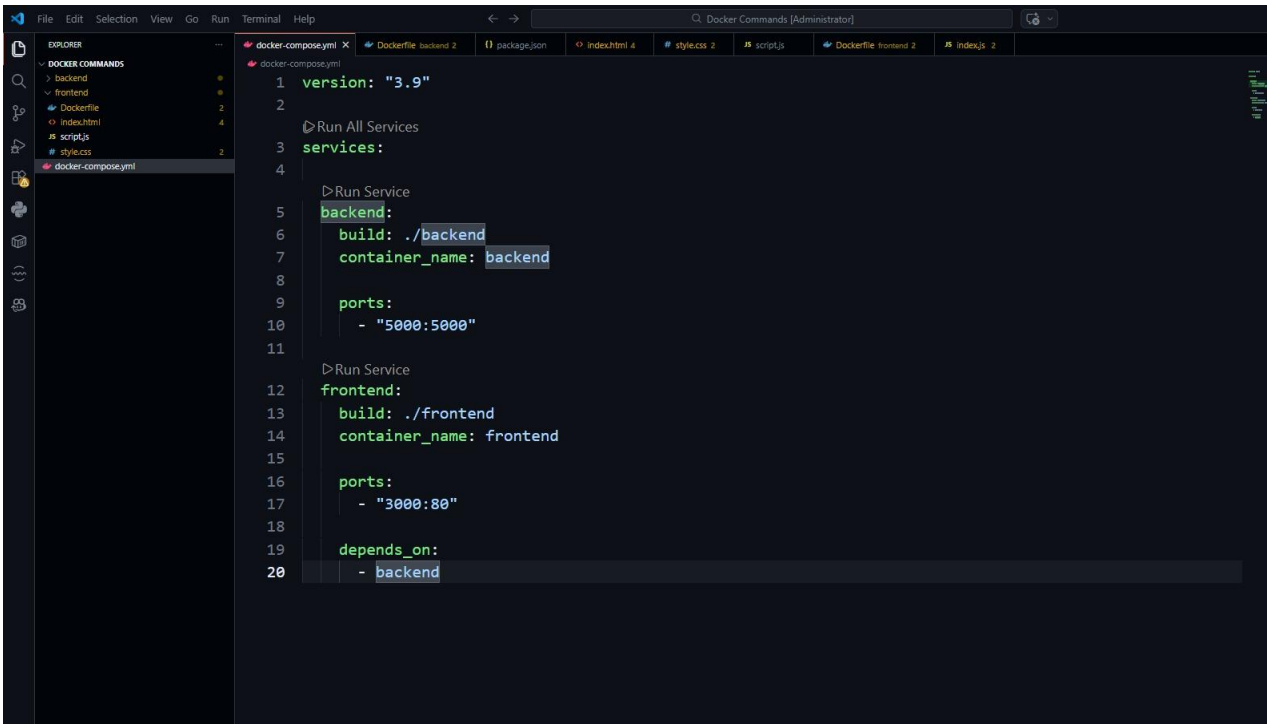


Frontend/Dockerfile:



Name: Arya Rane
Roll no.: 2401090
Batch: S21
Step 3:create docker-compose.yml

Docker-compose.yml:



```yaml
version: "3.9"

services:

  backend:
    build: ./backend
    container_name: backend

    ports:
      - "5000:5000"

  frontend:
    build: ./frontend
    container_name: frontend

    ports:
      - "3000:80"

    depends_on:
      - backend
```

Script.js



```javascript
async function calculate() {
    const num1 = Number(
        document.getElementById("num1").value
    );
    const num2 = Number(
        document.getElementById("num2").value
    );
    const operation =
        document.getElementById("operation").value;

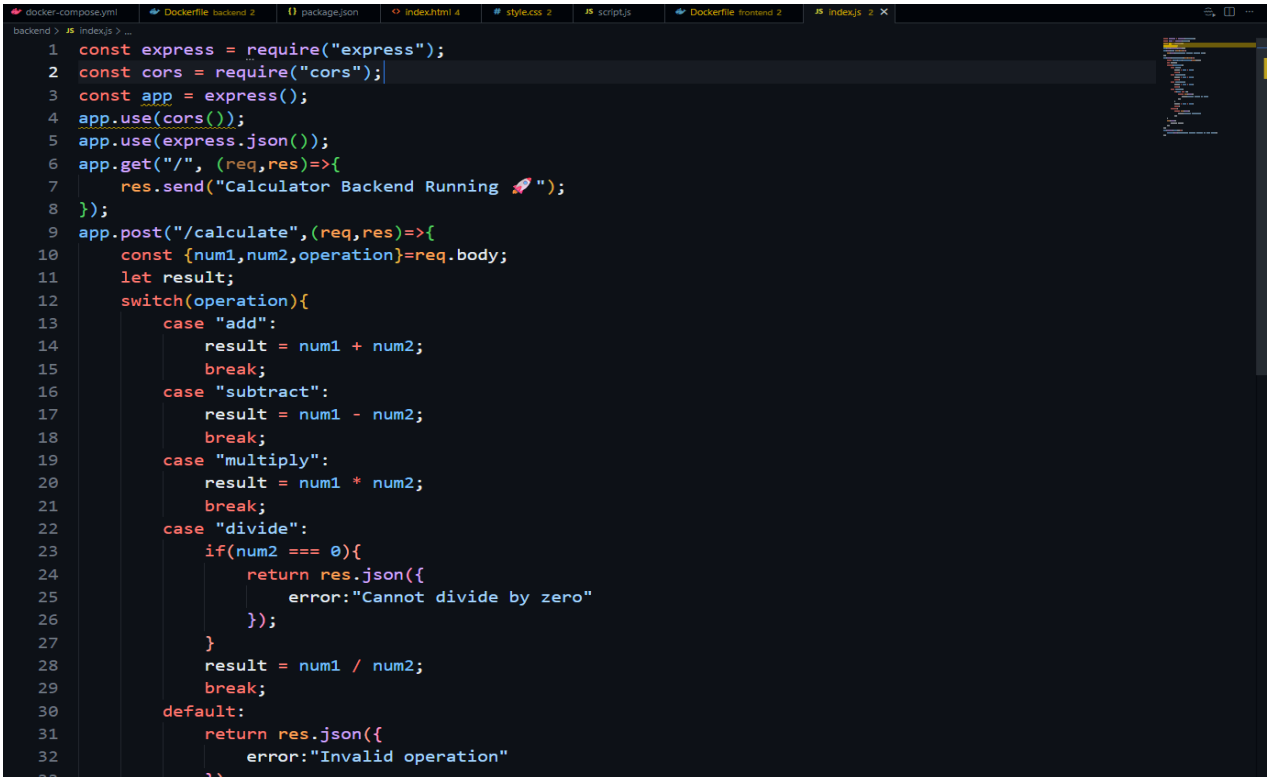
    const response = await fetch(
        "http://localhost:5000/calculate",
        {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },

            body: JSON.stringify({

                num1,
                num2,
                operation
            })
        });
    const data = await response.json();
    if (data.error) {
        document.getElementById("result").innerHTML =
            data.error;
    }
    else {
        document.getElementById("result").innerHTML =
            "Result: " + data.result;
```

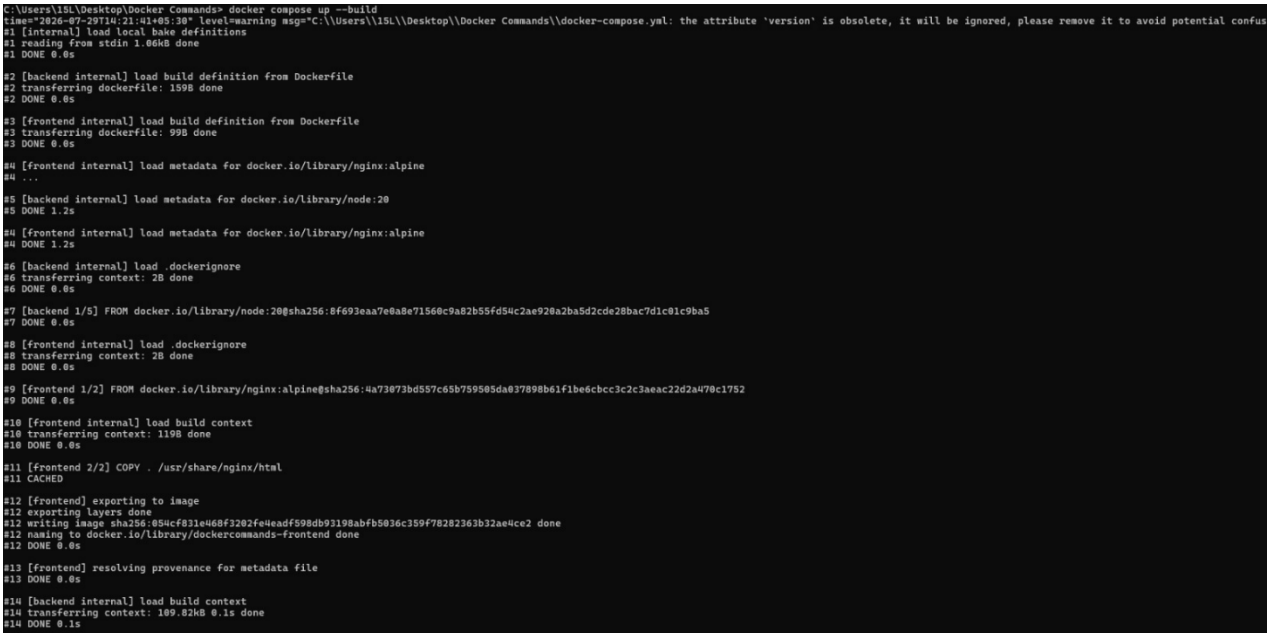
Name: Arya Rane
Roll no.: 2401090
Batch: S21

Index.js:



```
const express = require("express");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());
app.get("/", (req,res)=>{
    res.send("Calculator Backend Running 🚀");
});
app.post("/calculate",(req,res)=>{
    const {num1,num2,operation}=req.body;
    let result;
    switch(operation){
        case "add":
            result = num1 + num2;
            break;
        case "subtract":
            result = num1 - num2;
            break;
        case "multiply":
            result = num1 * num2;
            break;
        case "divide":
            if(num2 === 0){
                return res.json({
                    error:"Cannot divide by zero"
                });
            }
            result = num1 / num2;
            break;
        default:
            return res.json({
                error:"Invalid operation"
```

Step 4:build and start the services



```
C:\Users\15L\Desktop\Docker Commands> docker compose up --build
time="2026-07-29T14:21:41+05:30" level=warning msg="C:\\Users\\15L\\Desktop\\Docker Commands\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confus
#1 [internal] load local bake definitions
#1 reading from stdin 1.06kB done
#1 DONE 0.0s

#2 [backend internal] load build definition from Dockerfile
#2 transferring dockerfile: 159B done
#2 DONE 0.0s

#3 [frontend internal] load build definition from Dockerfile
#3 transferring dockerfile: 99B done
#3 DONE 0.0s

#4 [frontend internal] load metadata for docker.io/library/nginx:alpine
#4 ...

#5 [backend internal] load metadata for docker.io/library/node:20
#5 DONE 1.2s

#4 [frontend internal] load metadata for docker.io/library/nginx:alpine
#4 DONE 1.2s

#6 [backend internal] load .dockerignore
#6 transferring context: 2B done
#6 DONE 0.0s

#7 [backend 1/5] FROM docker.io/library/node:20@sha256:8f693eaa7e0a8e71560c9a82b55fd54c2ae928a2ba5d2cde28bac7d1c01c9ba5
#7 DONE 0.0s

#8 [frontend internal] load .dockerignore
#8 transferring context: 2B done
#8 DONE 0.0s

#9 [frontend 1/2] FROM docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c1752
#9 DONE 0.0s

#10 [frontend internal] load build context
#10 transferring context: 119B done
#10 DONE 0.0s

#11 [frontend 2/2] COPY . /usr/share/nginx/html
#11 CACHED

#12 [frontend] exporting to image
#12 exporting layers done
#12 writing image sha256:054cf831e468f3202fe4eadf598db93198abfb5036c359f78282363b32ae4ce2 done
#12 naming to docker.io/library/dockercommands-frontend done
#12 DONE 0.0s

#13 [frontend] resolving provenance for metadata file
#13 DONE 0.0s

#14 [backend internal] load build context
#14 transferring context: 109.82kB 0.1s done
#14 DONE 0.1s
```

Name: Arya Rane
Roll no.: 2401090
Batch: S21

```
#15 [backend 2/5] WORKDIR /app
#15 CACHED

#16 [backend 3/5] COPY package*.json ./
#16 CACHED

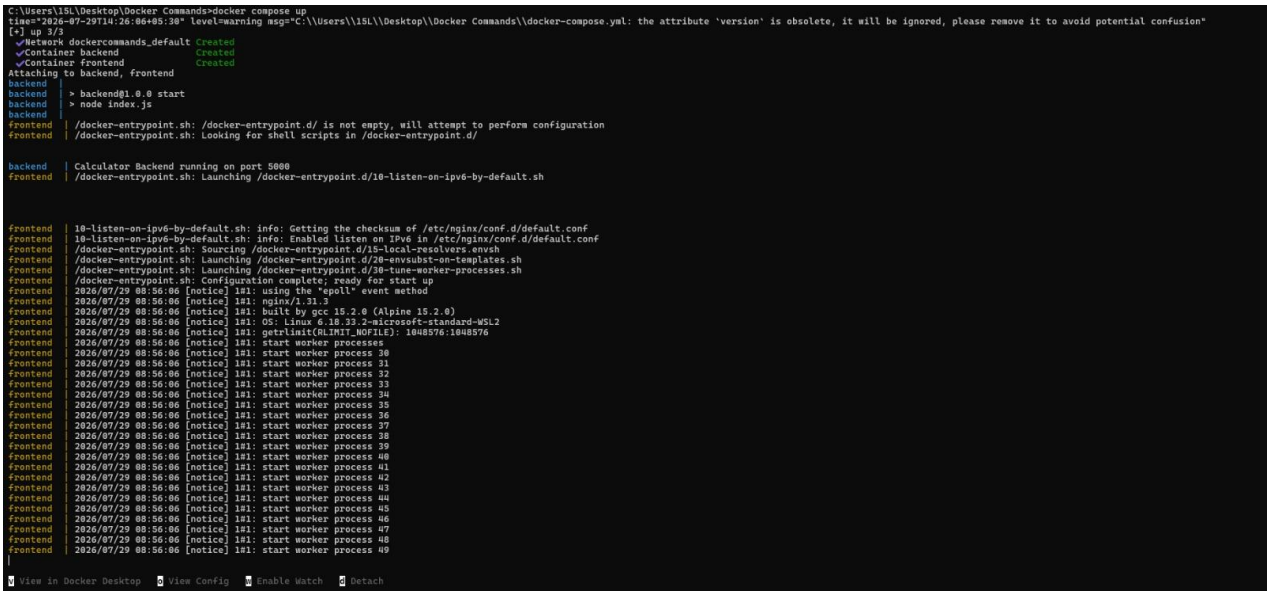
#17 [backend 4/5] RUN npm install
#17 CACHED

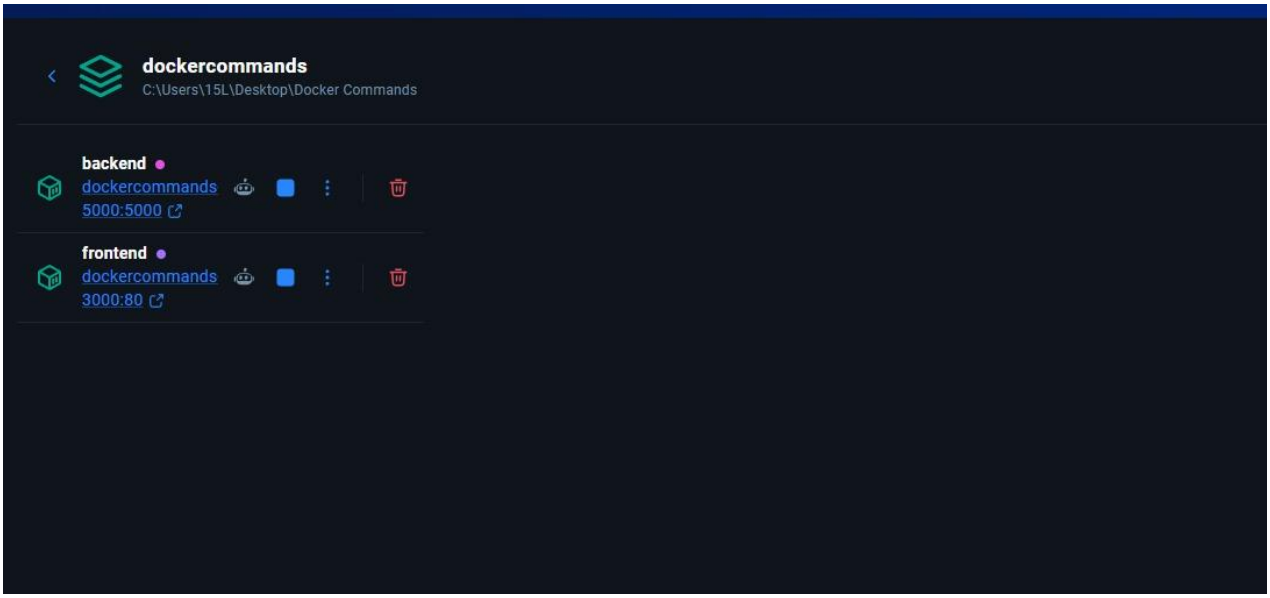
#18 [backend 5/5] COPY . .
#18 CACHED

#19 [backend] exporting to image
#19 exporting layers done
#19 writing image sha256:fcd302b5399c7db5a8d0dc46fd2e1144fe8ed2265febf55eee89c0e2e6b2278e done
#19 naming to docker.io/library/dockercommands-backend done
#19 DONE 0.0s

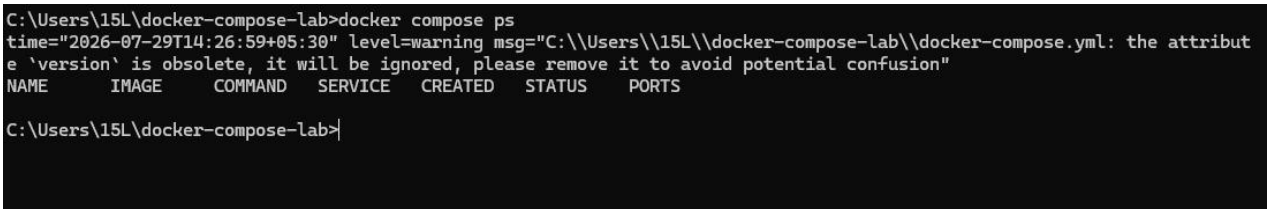
#20 [backend] resolving provenance for metadata file
#20 DONE 0.0s
[+] up 4/4
 ✔Image dockercommands-frontend Built                                                
```

Name: Arya Rane
Roll no.: 2401090
Batch: S21





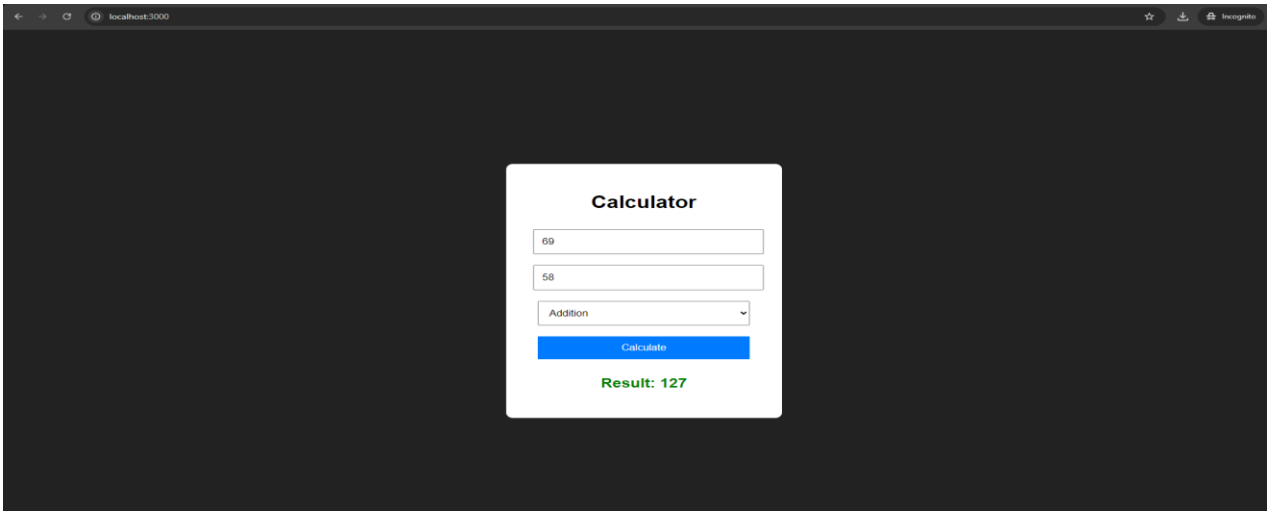
Step 5: verify the deployment



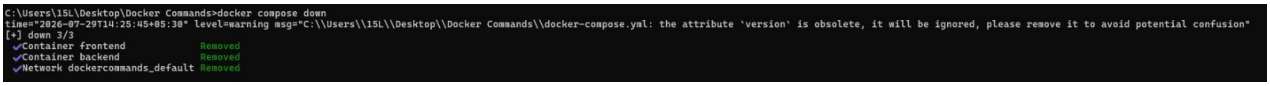
Name: Arya Rane
Roll no.: 2401090
Batch: S21

Step 6: open project on local host:5000





Step 7: stop and remove services



Conclusion:

Docker Compose provides an efficient and simplified approach for deploying and managing multi-service applications using Docker containers. By defining all application components in a single docker-compose.yml file, it eliminates the complexity of manually creating and configuring multiple containers. It automates tasks such as image building, container creation, networking, and service management, allowing different components of an application to communicate easily. Through this experiment, a multi-container application consisting of a frontend and backend service was successfully deployed using Docker Compose. The use of commands such as docker compose up --build and docker compose down demonstrated how Docker Compose helps in building, running, and managing applications consistently. Overall, Docker Compose improves development efficiency, ensures a reliable deployment environment, and provides a scalable solution for modern multi-service application development.